WATT-WATCH — 2-day plan for a 3-person team

Deadline: 2 days. This plan tells each person exactly what to do, step by step. You don't need to learn Git branching — only **Person 1** runs the code from Git; the other two download a ZIP.

Person	Role	Owns in the report and presentation
Person 1 (team lead)	Runs the app, database & billing	Database, tariff, bills, final report assembly
Person 2	Scanning & prediction	OCR (meter photos), prediction, early warning
Person 3	Appliances, slides & demo	Saving plan, screenshots, slides, demo video

Where the code is: https://github.com/ZumairaSultana/WATTWATCH → branch `claude/code-implementation-guide-fqx9ks` (On GitHub, click the branch dropdown that says `main` and pick this branch.)

Shared Google Doc: Person 1 creates one Google Doc called *"WATT-WATCH Report"*, shares it with both teammates (**Share → add their emails → Editor**), and pastes in these headings:

```
1. Abstract
2. Introduction (the problem)
3. Objectives
4. Existing system
5. Proposed system (WATT-WATCH and its flow)
6. System design (architecture, ER diagram, database tables)
7. Modules
   7.1 Database & billing        (Person 1)
   7.2 Scanning & prediction     (Person 2)
   7.3 Saving plan & interface   (Person 3)
8. Results
9. Limitations
10. Future work
11. Conclusion
12. References
```

DAY 1 — Morning (all 3 together, ~2 hours)

Step 1 — Person 1 installs and runs the app (Windows)

1. Install **Python** from python.org → Downloads. ⚠️ On the first installer screen **tick "Add python.exe to PATH"**, then **Install Now**.
2. Install **Git** from git-scm.com → Download for Windows (keep all defaults).
3. Press the **Windows key**, type **cmd**, press **Enter** (opens Command Prompt).

4. Type these lines one at a time, pressing **Enter** after each:

```
cd %USERPROFILE%\Documents
git clone https://github.com/ZumairaSultana/WATTWATCH.git WATT-WATCH
cd WATT-WATCH
git checkout claude/code-implementation-guide-fqx9ks
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python -m pytest
```

   - The last line must end with `32 passed`.
   - If `python` is "not recognized", use `py` instead of `python`.
5. Start the app:

```
streamlit run app.py
```

   The browser opens at **http://localhost:8501**. Keep the Command Prompt window open. To stop the app later: click the Command Prompt window and press **Ctrl + C**.

> Next time you open Command Prompt, you only need: `cd %USERPROFILE%\Documents\WATT-WATCH`, `venv\Scripts\activate`, `streamlit run app.py`

Step 2 — Teammates download the code (Persons 2 and 3, 5 minutes)

1. Open https://github.com/ZumairaSultana/WATTWATCH
2. Branch dropdown (says `main`) → pick **`claude/code-implementation-guide-fqx9ks`**.
3. Green **Code** button → **Download ZIP** → right-click the ZIP → **Extract All**.
4. Open the folder in VS Code or Notepad to **read** your files (you don't need to run them).

> Optional: if you want to run the app yourself too, follow Step 1 on your laptop.

Step 3 — Walkthrough (all 3, 30 minutes)

Person 1 shares their screen (or everyone sits together) and clicks through:

1. 🏠 **Households** — the 10 sample households from the database PDF.
2. Sidebar → select **H001 - Arun**.
3. 🧾 **Bills** → **Generate bill for this cycle** → a bill for 150 units appears.
4. 📷 **Scan & Track** → type **1480** → **Add reading**.
5. 📈 **Predict & Warn** — predicted units, bill and warning.
6. 💡 **Reduce & Save** — tick *Air conditioner*, *Water heater*, *Television* → saving plan.

Everyone should be able to explain this flow in one sentence: *"You enter meter readings, the app predicts the bill, warns before a costlier slab, and suggests which appliances to cut to save money."*

Step 4 — Each person reads their part (30 minutes)

Person	Read in `docs/GUIDE.md`	Read in `docs/TEAM_GUIDE.md`
Person 1	Step 1 (tariff), Step 8b (database)	Part 5 (database)
Person 2	Steps 2–5 (scan, track, predict, warn)	Part 7
Person 3	Steps 6–8 (appliances, saving plan, savings), Step 9 (app)	Part 8

DAY 1 — Afternoon & evening (separate work)

👤 PERSON 1 — Database & Billing (+ runs the app)

Step 1.1 — Check the tariff (1 hour) ⚠️ most important

1. Search for the **current Tamil Nadu domestic electricity tariff** on the official TNPDCL (Tamil Nadu Power Distribution Corporation) or TNERC (Tamil Nadu Electricity Regulatory Commission) website.
2. Open `wattwatch\config.py` in Notepad/VS Code and compare with `DEFAULT_SLABS`.
3. If a rate is different, change the number and save. Then in Command Prompt: `python -m pytest` (some tests use their own rates, so they should still pass).
4. Write in the report: the source website, the date you checked it, and the rates.

Step 1.2 — Bill check (30 minutes)

1. Get **2 old electricity bills** from home (paper or the online account).
2. For each bill, run (replace `450` with the units on the bill):

```
python -c "from wattwatch.tariff import Tariff, calculate_bill; from wattwatch impor
t config; print(calculate_bill(450, Tariff.from_rows(config.DEFAULT_SLABS)))"
```

3. Fill in this table in the report:

Bill	Units	Real amount (Rs)	App amount (Rs)	Difference	Possible reason

Step 1.3 — Database screenshots (45 minutes)

1. Install **DB Browser for SQLite** (free, sqlitebrowser.org).
2. **Stop the app first** (Ctrl + C), then **Open Database** → `Documents\WATT-WATCH\data\wattwatch.db`.
3. **Browse Data** tab → screenshot each of the 3 tables.
4. **Execute SQL** tab → run each query below → screenshot the result:

```
SELECT * FROM Household;

SELECT h.House_ID, h.Household_Name, SUM(m.Units_Consumed) AS Total_Units
FROM EB_Meter m JOIN Household h ON h.House_ID = m.House_ID
GROUP BY h.House_ID ORDER BY Total_Units DESC;

SELECT * FROM Electricity_Bill WHERE Payment_Status <> 'Paid';
```

5. **Close Database** in DB Browser before starting the app again.

Step 1.4 — ER diagram (30 minutes) Draw on **draw.io** (free, app.diagrams.net) — 3 boxes linked by lines:

```
Household (House_ID PK) ──< EB_Meter (Meter_ID PK, House_ID FK, Bill_ID FK)
Household (House_ID PK) ──< Electricity_Bill (Bill_ID PK, House_ID FK)
Electricity_Bill (Bill_ID) ──< EB_Meter (Bill_ID)
```

Export as PNG → put it in the report, section 6.

Step 1.5 — Add your 3 homes (30 minutes)

1. Ask Persons 2 and 3 for: last bill's **meter reading + date**, **today's reading**, meter number.
2. In the app: sidebar → ➕ **Add a household** → fill in → **Add household**. (Use made-up consumer numbers unless your family agrees to use the real one.)
3. 📷 **Scan & Track** → enter meter number, last bill reading + date, today's reading → **Add reading**.
4. 📈 **Predict & Warn** → screenshot each home's prediction.
5. Test Person 2's meter photos (Step 2.2) and send them the results.

Step 1.6 — Write report section 7.1 (evening) Explain: the 3 tables, primary/foreign keys, the 2 added columns and why (`docs/TEAM_GUIDE.md` Part 5), how a bill is generated, the tariff and the bill check table.

👤 PERSON 2 — Scanning & Prediction

Step 2.1 — Understand your part (1 hour) Read `docs/GUIDE.md` Steps 2–5. Write down in your own words:

- **OCR**: photo → grey → bigger → black & white → Tesseract reads digits → the longest number is the reading.
- **Prediction**: units per day = slope of a straight line through recent readings; predicted units = units so far + units per day × days left.
- **Warning**: if predicted units go past a slab limit (100, 400, 500 …), warn how many days until it's crossed and how many units to cut.

Step 2.2 — Meter photos (1 hour)

1. Take **10 photos** of meter displays (your home, neighbours, relatives — with permission). Mix them: 4 clear, 3 dim light, 3 at an angle.
2. For each photo, write down the **real reading** you can see.
3. Send the photos and real readings to Person 1 (WhatsApp/Drive).
4. Person 1 uploads each one in 📷 **Scan & Track → Read number from photo** and tells you what the app read. (This needs Tesseract installed on Person 1's laptop — search "Tesseract UB Mannheim" for the Windows installer.)
5. Fill in:

Photo	Real reading	App read	Correct?	Light / angle

6. **Accuracy = correct ÷ 10 × 100.** Example: 6 correct → 60%. A low number is fine if you explain it — it's why the user can correct the reading.

Step 2.3 — Your home's prediction (30 minutes) Give Person 1 your last bill's reading + date and today's meter reading. From their screenshot, note: units so far, units per day, predicted units, predicted bill, warning message.

Step 2.4 — Write report section 7.2 (evening) Explain OCR, prediction and warning (use your notes from 2.1), the OCR accuracy table, and your home's prediction. Add to **Limitations**: OCR struggles with glare/angles; the prediction assumes steady daily use; real bills had not arrived yet to compare.

👤 PERSON 3 — Appliances, Slides & Demo

Step 3.1 — Understand your part (1 hour) Read `docs/GUIDE.md` Steps 6–8. Write down in your own words:

- **Energy per day** = watts × quantity × hours ÷ 1000 (units/kWh).
- **Saving plan**: cut hours from the most powerful appliances first, at most 50% of each, never the "essential" ones (fridge).
- **Savings** = bill before − bill after, and CO_2 = units saved × 0.71 kg (approximate).

Step 3.2 — Appliance wattages (45 minutes) At home, read the **sticker/label** on 5–6 appliances (AC, geyser, fridge, fan, TV, motor). Fill in:

Appliance	Watts on label	App's typical watts	Hours used per day

If a label shows only amps (A): watts ≈ 230 × amps (rough). Send the table to Person 1.

Step 3.3 — Saving plan example (30 minutes, with Person 1) Person 1 opens 💡 **Reduce & Save** for your home, ticks your appliances and enters your hours. Screenshot the saving plan and savings. Note: which appliances to cut, units saved, money saved.

Step 3.4 — Slides (2 hours) Make ~12 slides (Google Slides or PowerPoint):

#	Slide	Who presents
1	Title: WATT-WATCH, team names, guide name	Person 1
2	Problem: slab billing, bill shock	Person 1
3	Solution & flow: Scan → Track → Predict → Warn → Reduce → Save	Person 1
4	Technology: Python, Streamlit, SQLite, Tesseract OCR	Person 1
5	Database: ER diagram & tables	Person 1
6	Bill calculation & tariff check	Person 1
7	Scanning (OCR) & accuracy	Person 2
8	Prediction & early warning	Person 2
9	Saving plan & savings	Person 3
10	Live demo	Person 3
11	Results, limitations, future work	Person 3
12	Conclusion / Thank you	Person 3

Step 3.5 — Write report section 7.3 (evening) Explain the appliance formula, the saving plan method, your example, and the app's 5 tabs (with screenshots).

DAY 2 — Finish & practise

Time	Who	Task
9:00 – 11:00	Person 1	Write sections 1–6 and 11 of the report; insert all screenshots
9:00 – 11:00	Person 2	Write sections 8 (Results) and 9 (Limitations)
9:00 – 11:00	Person 3	Finish slides; write section 10 (Future work)
11:00 – 13:00	All	Read the whole report together; fix mistakes; same font and headings
14:00 – 15:00	Person 3 + 1	**Record a backup demo video** on Person 1's laptop (Windows: **Win + G** → record) following the demo script below
15:00 – 17:00	All	**Practise the full presentation 3 times**
17:00 – 18:00	All	Practise the questions below; everyone answers every question once

Demo script (Person 3 presents, Person 1 clicks)

1. **Households** — "These are households stored in our SQLite database."
2. Select your home → **Scan & Track** — upload a meter photo → **Read number** → **Add reading**.
3. **Predict & Warn** — "We will use about X units; the app warns us before we cross Y units."
4. **Reduce & Save** — tick appliances — "Cutting the AC by 1 hour a day saves Rs Z."
5. **Bills** — generate a bill → mark it Paid.

Questions examiners may ask (everyone should know all answers)

Question	Answer
Is it an app or a website?	A web application: runs in any browser, built with Python + Streamlit, data in SQLite.
Why not a mobile app?	A web app works on every device with a browser and was faster to build; a mobile app is future work.
How does prediction work?	Units per day from a straight line through recent readings × days left in the cycle.
Why cut the biggest appliances first?	One hour of a 2000 W geyser saves as much as 20 hours of a 100 W TV.
What if OCR reads the wrong number?	The user sees the number and corrects it before saving.
Why SQLite?	Built into Python, one file, no server needed — enough for this app.
Why calculate units instead of typing them?	Prevents mistakes: units = current − previous reading.

Question	Answer
Is the data real?	The 10 households are fictional sample data; their dates are assumed. Our own homes are real readings.
What if the tariff changes?	Rates are in `config.py` and can be edited in the app's sidebar.
Did you compare with real bills?	Old bills were checked against the bill calculator; the full-cycle prediction will be compared when the next bills arrive.

On presentation day

- Start the app **before** the presentation (`streamlit run app.py`).
- Keep the **backup video** and the slides on a pen drive too.
- Laptop charged; open the app tab and the slides in advance.

If something goes wrong

Problem	Fix
`python` not recognized	Reinstall Python with **Add to PATH** ticked, or use `py`
`(venv)` missing at the start of the line	`venv\Scripts\activate`
PowerShell says scripts are disabled	Use **Command Prompt** (cmd), not PowerShell
`ModuleNotFoundError`	Activate venv, then `pip install -r requirements.txt`
`database is locked`	Close DB Browser, restart the app
Want fresh sample data	Sidebar → **Database** → tick *I understand* → **Reset to sample data**
Photo scan says Tesseract not found	Type the reading instead (the app still works); mention it in the demo
Anything else	Copy the exact error message and ask for help